EEE3032 - Computer Vision & Pattern Recognition
Coursework Assignment Report

Visual Search of an Image Collection

Autumn 2025

Lampros Grammatikopoulos

6918674

lg01302@surrey.ac.uk

Abstract

This coursework presents the creation and assessment of a visual search system for retrieving images depending on visual similarity. The dataset that was used was part of Microsoft's research named "MSRC-v2 image dataset" and contained 591 images divided into 20 object categories. The implemented system extracts image descriptors to represent visual characteristics of images, compares them using defined distance metrics, and ranks images according to similarity to a given query image. This coursework was implemented using the Python language.

A global colour histogram was first implemented as a baseline descriptor, utilizing varying levels of RGB quantization and Euclidean distance for similarity measurement. Performance was evaluated using precision-recall statistics, mean average precision, confusion matrices, and the query result themselves providing quantitative and qualitative insights for the retrieval effectiveness. The system was further extended with spatial grid-based descriptors combining colour and texture information and their performance was compared. Afterwards, PCA-based dimensionality reduction was introduced using Euclidean and Mahalanobis distances and evaluation of PCA results was made. Lastly, twelve difference distance metrics were compared in terms of effectiveness using mean average precision to assess their impact on accuracy and efficiency.

The results of the experiments demonstrate that combining spatial and texture information together significantly improves retrieval performance while adding color information can also help build a more robust visual search system. The PCA technique was analyzed and tested, showing computational benefits with minimal accuracy loss and accuracy improvement on large dimensions. In addition, experiments showed that the best distance measures for the system were Bhattacharyya, Hellinger, Chi-Square, Manhattan and Intersection. The findings underline the effect a descriptor design and similarity measure choice can have in visual search system.

Table of Contents

Global Colour Histogram (GCH)

The development and evaluation of this image retrieval system based on visual similarity was created using a dataset from Microsoft's research named "MSRC-v2 image dataset" that contained 591 images divided into 20 object categories

One way to represent the contents of the images was to find the overall colour distribution of images and store them. Thus, a global colour histogram was implemented which extracts image descriptors to represent visual characteristics of images. Specifically, a colour histogram can be displayed as a bar graph which visualizes the distribution of all the pixel values that an input image holds. Specifically, this bar chart contains different regions which hold the values of each RGB colour channel. These regions are called bins and by splitting the RGB channels, they count how many pixels belong in them. The x-axis of the histogram symbolizes these RGB bins, while the y-axis indicates the frequency of pixels that are stored in them [1]. The number of bins determines the quantization level (e.g., 8-level quantization on RGB has 8x8x8=512 bins) and is crucial when creating colour histograms [2]. Furthermore, higher number of bins offer finer color levels and bin matching, however they as the number of bins grow the computational complexity is increased. In contrast, less bins offer computational efficiency but may fail to capture and store colour detail that is often important [1].

After having extracted all the information from the images it was necessary to query one image over the entire dataset. To do that, a comparison between the histograms that represented the images was needed [3]. This was possible by using the Euclidean distance measure which is one of the measures using to compare histograms [4], [5].

Calculation procedure

To match images using their color distribution global color histogram were calculated by running *cvpr_computedescriptors.py*. Firstly, given an input image, each pixel was analyzed using the RGB channels of the image. These channels were divided into a smaller number of bins using *compute_color_histogram()* which applied *np.floor(img * bins_per_channel)* and calculated the $indices = r * bins\text{^}2 + g * bins + b$ that combined R, G, B channels for all pixels. Afterwards, a histogram for the image was formed by using *np.bincount* which analyzed these indices and counted the number each color bin. Then, the histogram was normalized so that it summed to one, and the image descriptor was extracted. The normalized color histogram of the descriptor, represented a color feature vector of probabilties for each bin. This vector was a single point in a color space where each axis is a color bin. These vectors were then used to compare two images, using the Euclidean distance between them, which is defined as $D(A,B) = \sqrt{\sum_{i=1}^{N}(h_{A,i} - h_{B,i})^2}$. This distance was implemented in *cvpr_compare.py* using np.linalg.norm(F1 - F2) which computed the Euclidean distance [6]. This distance measured how different the two color distributions were. Specifically, smaller values meant that the images had similar color distributions, while larger values showed they were different. This procedure was an effective way of estimating visual similarity based purely on overall color content.

Insights

After applying with different quantization levels (q=4,8,12,16,20,24,28,32) per RGB channel to the system, the total bins for each quantization level were averaged for all images and extracted as shown below in Figure 1. The plot reflects that as the number of bins increases, the total probability (which is the likelihood that a randomly chosen pixel from an image belongs to a specific color bin) gets distributed across more bins, so the average normalized frequency per bin decreases proportionally, resulting in finer color resolution but lower individual bin values overall.

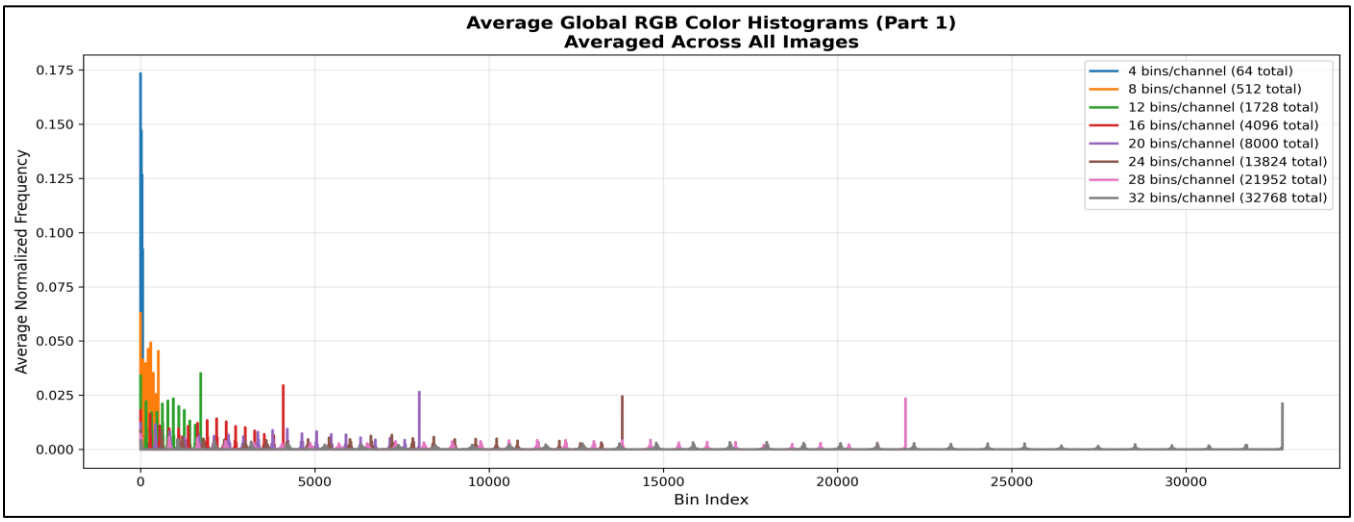

Figure 1: Global Colour Histogram, Colour Frequency - Bins

By running *cvpr_visualsearch.py*, *a random query image gets selected from the dataset and compared to each descriptor thus each image in the dataset*, the below results were taken (Figure 2 and Figure 3):

Figure 2: Results of random queries for different q=4,8,12,16.

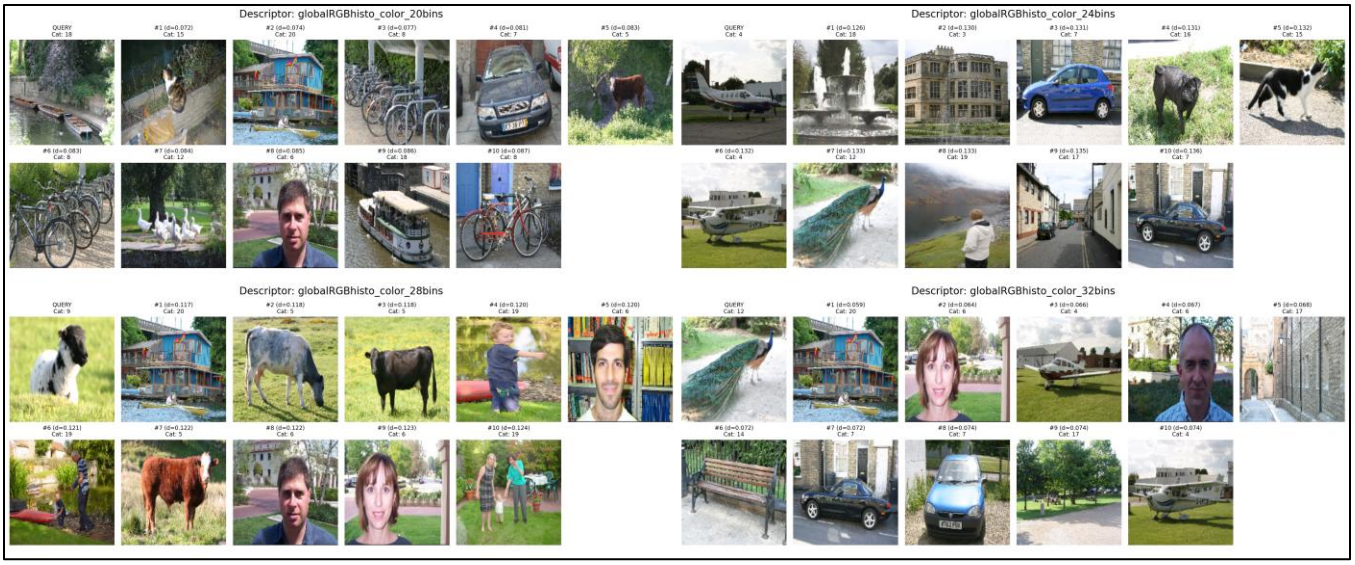

Figure 3: Results of random queries for different q=20,24,28,32.

The above results in Figure 2 and Figure 3, show that the best results are acquired when setting the number of color bins to 4 or 8. In the top 10 results, 4 out of 10 are correctly matched and for q=8 the 3 out of 4 correct results are also on the top 3 positions, but increasing the bins has a negative effect in the system's accuracy since wrong classes are retrieved. One the other hand, if only the similarity between images is required (same colors: e.g. images with grass are full of green pixels), then different bins can work as well (e.g. when q=28 in Figure 3 grass with cows are retrieved instead of grass with sheep).

However, these queries are randomly selected and are only some of the potential queries available. For a correct evaluation of the descriptors, it is necessary to test them using the same query for proper and exact comparison. Thus, a query from class 2, which contains images of trees, is selected for this purpose as displayed in Figure 4 and Figure 5 below.

Figure 4: Results of the same query image for different q=4,8,12,16.

Figure 5: Results of the same query image for different q=20,24,28,32.

By evaluating Figure 4 and Figure 5 comparing the top-10 results, it is clear that when q=4 there are 4 wrong images, when q=8 there is only 1 wrong image, when q=8 or q=16 there are 2 wrong images, and when q=20 or q=24 there are 3 wrong images. This evaluation confirms that choosing 8 color bins (q=8) is the best option for this class between all descriptors tested. However, this evaluation needs to be repeated for every class across the dataset to confirm this decision and determine the most successful q value in general for the system.

To see the above results, run the files: *cvpr_computedescriptors.py* and *cvpr_visualsearch.py* by only selecting and using *globalRGBhisto_color_**X**bins* descriptors.

Challenges

One of the most important actions for image retrieval systems is to be built for generalization. However, to achieve that, an accurate dataset must be assembled for the system to be trained on and achieve high results. Inconsistencies on the labels or classes of images in the dataset can cause issues with the system's accuracy. It has been identified that some of the images in the dataset are misplaced in a different class e.g. 16_22_s.bmp contains humans but is placed on dog class instead of class 19 (human class), also some goat images e.g. 1_26_s.bmp and 9_3_s.bmp are placed in wrong classes. This can have a negative effect for color descriptors, since goat images contain both the goat (white-gray color) and the grass (green color). However, descriptors that account for both color and spatial information in the image can improve them with generalization.

Evaluation methodology

To evaluate the system that uses global colour histogram in depth, some performance metrics can be used [7]. These metrics include the precision, which is the fraction of relevant images from the fetched ones, and the recall, which is the fraction of relevant images that were retrieved in total, [8]. They are calculated as follows [9]: *Precision = (Similar retrieved images) / (All **retrieved** images)*, *Recall = (Similar retrieved images) / (All **similar** images)* [10], and were used to evaluate how accurate the system was when querying an image over the dataset.

For enhanced analysis the precision-recall curve (PRC) was used, which represents the comparison between these two metrics simultaneously. A high area under this curve means that both recall and precision achieve high values. High precision is reached by having few irrelevant images marked as relevant in the returned results, and high recall is reached by having few relevant images that were marked irrelevant in the relevant results. If both metrics achieve high scores then that means the system is returning relevant images as well as returning most of the relevant images [11].

Another metric that was used to determine how effective the system was, is called Mean Average Precision (mAP). The mAP metric is produced by using Average Precision (AP). The AP metric represents the total area below the precision-recall curve and can be computed using the following equation: $AP = \frac{1}{RI}\sum_{x=1}^{RI} P(x)rl(x)$, [12]. Specifically, RI represents the count of relevant images, $P(x)$ is the precision at x, and $rl(x)$ shows if the x^{th} retrieved image is related to the query image ($rl \in \{0,1\}$). If the calculation for AP is repeated for each query, then the result is the mean average precision (mAP) across these queries, and it is the average of the calculated APs [13]. This methodology has been applied to this system to further evaluate the system's overall accuracy and robustness across eight different descriptors.

Furthermore, evaluating only the top *k (in this system top 10)* retrieved results is justified in ranked retrieval tasks where users typically examine only a few top results [14], [15]. Thus, the use of truncated measures such as mAP@K (e.g., mAP@10) can be applied in this top 10 retrieved images evaluation.

In image retrieval research, evaluation of pure similarity-based retrieval (i.e. without using category labels) is typically done via precision–recall curves and mean average precision (mAP), since the task is to retrieve relevant items from a database [16]. By contrast, when retrieval is constrained by known categories (i.e. predicting class labels), the task becomes a supervised classification problem, and evaluation is made using predicted probabilities via a confusion matrix, from which class-wise precision, recall, accuracy, and error rates can be computed [17]. In this system it is required to check the similarity of an image across many classes, thus confusion matrices will be used. This is a matrix of size C×C (where C is the number of classes) with actual classes of images on one axis and predicted classes of images on the other [18]. The sum of the numbers in all the cells gives the total number of images evaluated. Further, the correct predictions are the diagonal elements of the matrix [19].

Calculation procedure

The system generates plots for the top 10 query results, different PRC plots and confusion matrices. Specifically, *display_query_results()* shows query retrieval results with distance scores,

plot_confusion_matrix() creates normalized confusion matrices from top-1 predictions, and *plot_map_pr_curves_per_class()* plots per-class precision-recall curves computing mAP@[1-max] by averaging all precision values across full recall range. In addition, *plot_ap_one_query_pr_curves_per_class()* visualizes single-query AP calculations per category, *plot_combined_pr_curves()* compares descriptors showing both full mAP@[1-max] curves and mAP@[1-10] computed from k=1-10 precision averages, and lastly *plot_pr_curve_for_query_with_params()* compares all descriptors for a single query with AP scores.

Insights

After running experiments, the below Table 1 is computed. Table 1 contains the average precision of the top 10 results, as well as the mAP of all results, of the image query for each different quantization levels that were used (q=4,8,12,16,20,24,28,32):

Global RGB Histograms – 4/8/12/16/20/24/28/32 bins		
Descriptor	**mAP@K[1–10]**	**mAP@K[1–max]**
4bins	0.268	0.157
8bins	0.315	0.166
12bins	0.314	0.162
16bins	0.308	0.155
20bins	0.302	0.150
24bins	0.298	0.146
28bins	0.292	0.142
32bins	0.286	0.139

Table 1: mAP for top 10 results across all queries and mAP for all results across all queries.

By reviewing the above Table 1 it is clear that moving from 4 quantization levels to 8, noticeably increases the performance of the system as the mAP for all results (mAP@K[1-max]) increases. Furthermore, adding more quantization levels makes the system less accurate while also increasing the descriptor size the computational cost (16 bins/rgb channel = 16x16x16 bins = 4,096 bins for each descriptor!) also increases rapidly. In addition, when looking at the performance of the average top 10 query results, the same conclusions apply here as well, since mAP@K[1–10] does not offer that much better results after 8-12 bins.

Below are two plots in Figure 6 displaying the Precision-Recall curves (PRCs) which are calculated using q=8. Specifically, the left plot shows the PRCs of a random query for every class available along with each class AP value. The right plot displays the PRCs that are calculated from the mean of all queries for each class, displayed together with the averaged mAP curve which is calculated using all of them (the black dashed curve), and shows the mAP value for each class. This black dashed curve is used in the next two plots in Figure 7 further below to showcase the mAP of all classes combined for each descriptor for mAP@[1-10] and mAP@[1-max].

Specifically, results are shown for the top 10 images retrieved (left) and for all images retrieved (right) in the experiments. This visually displays that increasing the bins after a certain point (which here seems to be q=8), no improvement can be achieved. Since, the area under the curve decreases after q=12 and above. Consequently, increasing the number of bins, and thus the dimensionality, beyond this point would only make the system slower without adding significant value!

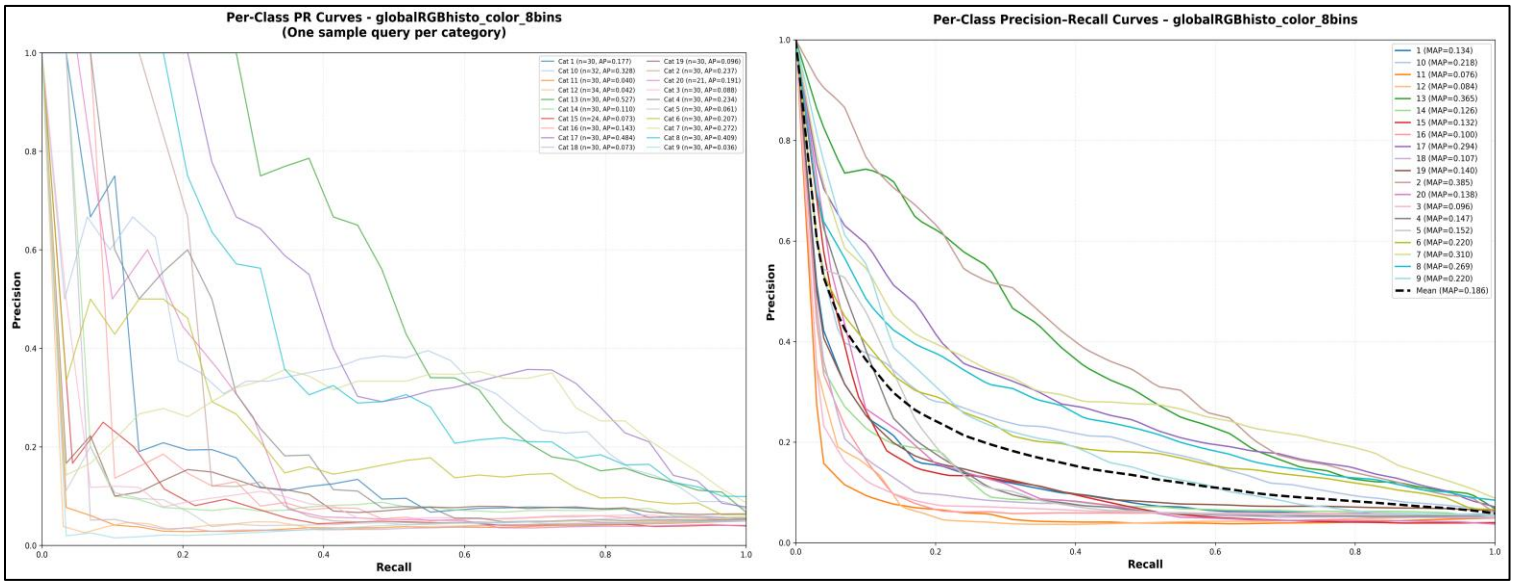

Figure 6: PRC of one random sample for every class (left) - PRC of every query for each class with their mean mAP curve (right), both using 8 color bins

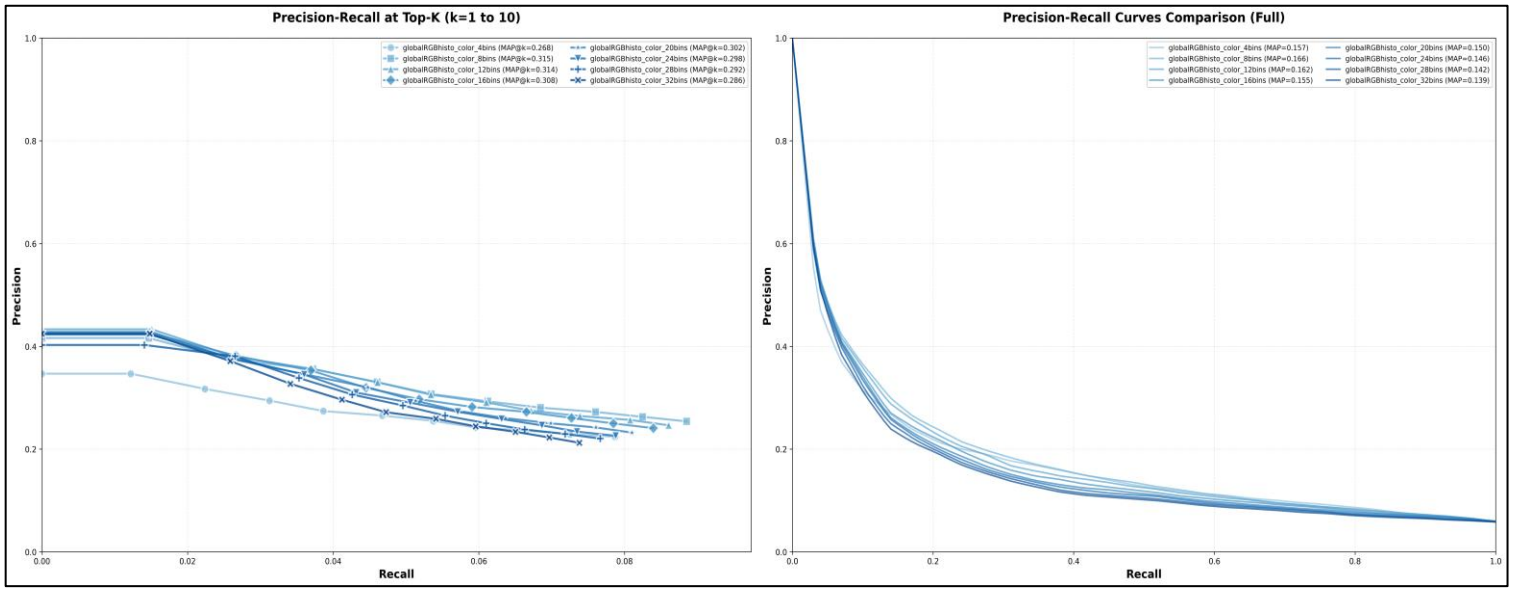

Figure 7: Comparison of averaged mAP of every class between all descriptors for top 10 results (left) and for all results (right), both using 8 color bins.

In addition, since each query image is part of a class (tree, sheep, cow, ... etc.), the evaluation of the system can also be made using predicted probabilities via a confusion matrix. Therefore, for further system analysis, three Confusion Matrices along with their related query results are plotted for different quantization levels q=4,8,12 for comparison in Figure 8, Figure 9, and Figure 10. Between the 3 queries, the best retrieval results are clearly when q=8, since 4 out of 10 images are retrieved correctly for class 9 (sheep) and 3 of the 4 results are retrieved before any wrong results (in top positions) indicating the descriptor had good accuracy. This can also be confirmed by viewing the confusion matrix column and row for class 9 and verifying that the system had 70% correct predictions in the total results. On the other

hand, when q=4 the correctly retrieved results are again 4 out of 10, but in between them there were many irrelevant images retrieved indicating the descriptor had less accuracy, thus the confusion matrix shows only 57% correct predictions for class 9. Lastly, when q=12 the performance was again not good for class 16 (dogs) with only 1 relative image being retrieved in the top 10 in the 2nd position, thus the confusion matrix shows only 40% correct predictions.

In conclusion it is noticeable by the confusion matrix diagonals that some classes consistently outperform other classes in all descriptors of the system due to their images color palette (by having more same colors e.g. class 2 – grass, has many green pixels and achieves 87% for q=8) thus achieving higher results with this global color histogram descriptor. Furthermore, the confusion matrix results are consistent to the PR curves results since the accuracy improvement stops at q=8!

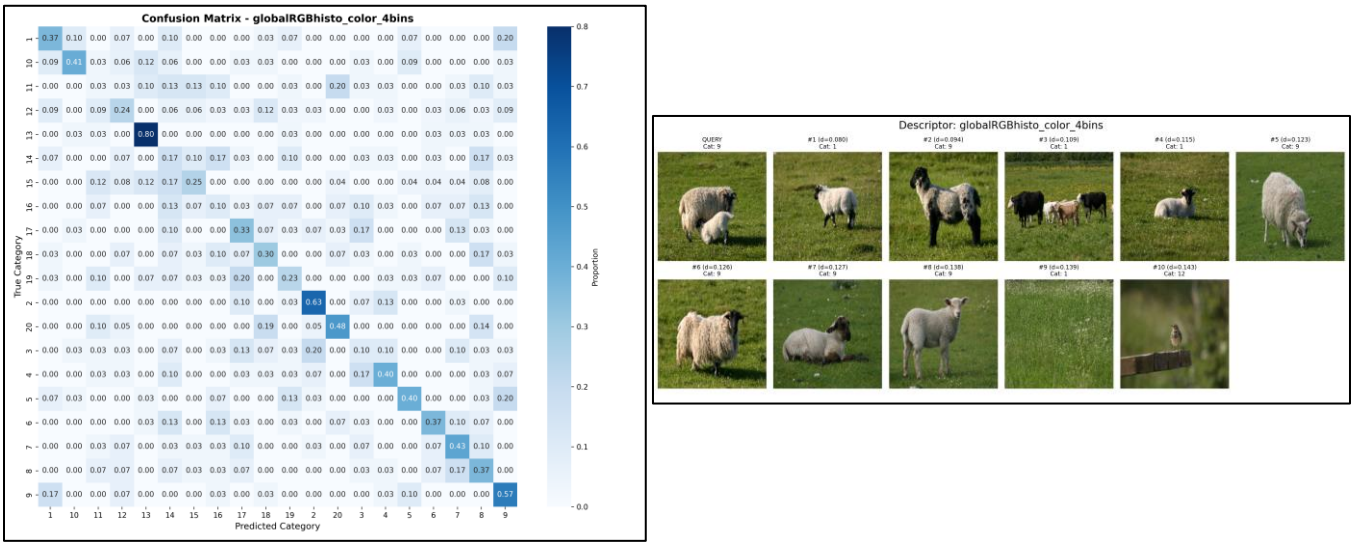

Figure 8: Confusion Matrix and related query results for q=4.

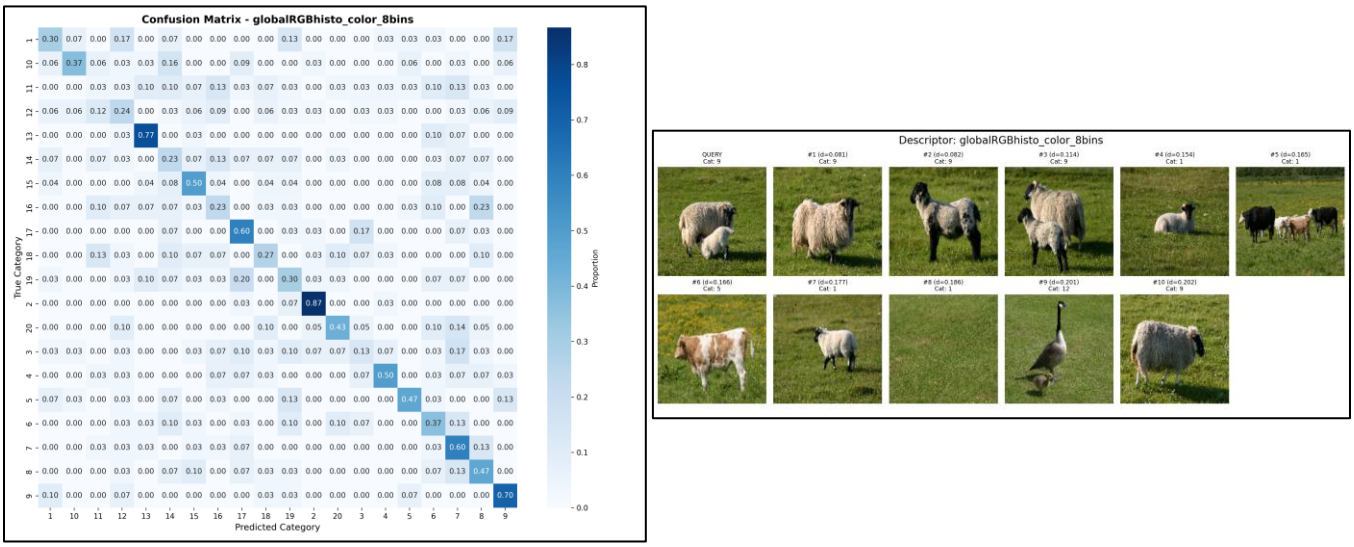

Figure 9: Confusion Matrix and related query results for q=8.

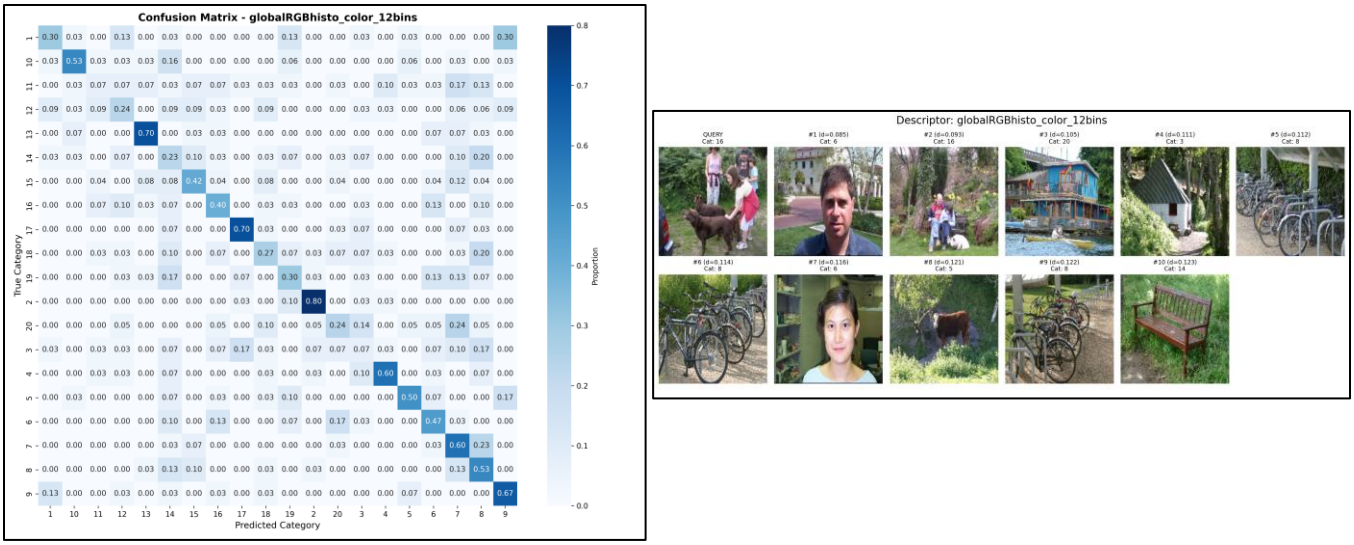

Figure 10: Confusion Matrix and related query results for q=12.

Spatial Grid and Edge Orientation Histogram (Colour and Texture)

Global color histogram was a usefull technique, though it only acounted for the colour of an image. This is not enough to retrieve images based on a query image from a dataset with complete acuracy. One way to improve on this problem is by using a technique called Spatial Grid to account for texture as well. Spatial Grid is different from the global colour histogram since it arranges all image cells into a single one dimensional vector, where each column corresponds to a separate dimension, thus forming a new feature space. In this approach, spatial information is preserved, since the values of individual cells for each color channel are retained rather than being grouped into a single value like in rgb histrograms. This technique can also be combined with the global color histogram for better results.

In addittion, the process of selecting the number of grids is crucial. In detail, a large number of grids can lead to a very sparse feature space causing reduce effectiveness in the system. In contrast, selecting only a small number of grids will lower the system;s capability to correctly discriminate images since only a few features will be kept for analysis. Therefore, choosing the number of grids is a task that only gets solved when testing values experimentally to figure out the best one [20].

One approach to further increase the perfomance of the system, is by accounting for localized regions (edges, corners, contours) of the image allowing features to be computed per region so that spatial relationships and layout are preserved in the overall feature representation of the computed descriptors. This can be achieved using an Edge Orientation Histogram (EOH), that is computed within each grid cell to capture local structural information [21]. The Edge Orientation Histogram (EOH) is a local descriptor that computes histograms of gradient directions within image cells, capturing local structure such as edges, corners, and contours [22]. The gradients are computed at each pixel by convolving Sobel masks with the image. Therefore, it is useful to examine how well this technique works in combination with the spatial grid technique and with or without the global color histogram.

Calulation procedure

Computing the spatial grid and edge orientation histograms, started by diving each image into a grid of equally sized cells (e.g., 2×2, 3×3, 4×4). For color features, a Global Color Histogram (G.C.H.) was applied as previsouly discussed in Global Colour Histogram (GCH) section but instead of passing the entore image to the *compute_color_histogram()*, a smaller image cell was given as input. Then, to detect edges, the Edge Orientation Histogram (EOH) was computed in *compute_texture_histogram()* by first converting each cell to grayscale since detection relies on intensity changes, and not color information like the in G.C.H.. Next, the gradients $grad\_x$ (horizontal change in intensity) and $grad\_y$ (vertical change in intensity) for both the x and y axes were produced using Sobel operators from the OpenCV library [23]. From these gradients, the gradient magnitude ($\sqrt{(\text{grad\_x}^2 + \text{grad\_y}^2)}$ and orientation $\arctan2(\text{grad\_y}, \text{grad\_x})$ were computed at each pixel. Then, orientations were mapped to the [0, 2π] range to even out bin distributions for gradient angles and quantized into a fixed number of orientation bins (e.g., 4, 8, 12, 16). In addition to the above, each orientation bin was weighted by the corresponding gradient magnitude, emphasizing strong edges while de-emphasizing weak or noisy gradients, similar to

standard HOG descriptors [24]. The resulting histogram for each cell was then normalized to one, and was able to capture local structural information such as edges, corners, and contours.

Finally, both color and texture features from all cells were concatenated in *spatialGridHistogramDescriptor()* to form the final spatial grid descriptor, preserving spatial relationships while combining appearance and structure information.

Insights

Spatial grid, E.O.H, G.C.H., and their combinations were put into test by executing 46 different experiements and comparing them using mAP PRC plots. The descriptors used in experiments consisted of different combinations of grid sizes (2x2, 3x3, 4x4), quantization levels (q=4,8,12,16 and 20,24,28,32 for color only), and different levels of angular quantization (orientation bins: 4,8,12,16).

The results show that a system that combines spatial grid with either global color histogram, or edge orientation or both of them, significantly outperforms a system that only accounts for color without looking at edges or applying gridding! Specifically, when comparing all the descriptors that used each of these techniques, it's easily detectable from the mAP PRC plots in Figure 11 that the spatial grid technique combined with edge orientation historgam is the best combination for when evaluating the overall perfomance of the system. In addittion, the spatial grid technique combined with both edge orientation historgam and global color histogram is best when evaluating the top-10 retrieved results. Lastly, applying gridding in global color histogram surpases global color histogram alone.

The Table 2 below is a summary of the mAP of the best descriptors from Figure 11.

Best Descriptors	Description	mAP@K[1–10]	mAP@K[1–max]
globalRGBhisto_color_8bins	Global Color Histogram (Euclidean)	0.315	0.166
3x3_color_4bins	Spatial Grid (3×3) + Global Color Histogram (Euclidean)	0.342	0.183
3x3_texture_16orient	Spatial Grid (3×3) + Edge Orientation Histogram	0.351	0.204
3x3_both_4bins_8orient	Spatial Grid (3×3) + Global Color Histogram (Euclidean) + Edge Orientation Histogram	0.366	0.194

Table 2: mAP for G.C.H. vs Spatial Grid & G.C.H. vs Spatial Grid & E.O.H. vs Spatial Grid & G.C.H. & E.O.H.

Furthermore, by reviewing Figure 11, it is clear that the best descriptor was the 3×3 spatial grid with 16 orientation bins achieving MAP = 0.204. The 3×3 grid size was always the leading choice across all descriptors that had spatial grid applied, as it represented the best balance between spatial localization and generalization. The 2×2 grids were too wide to capture adequate spatial structure, while 4×4 grids were too localized since they over-segmented images into very small chunks, leading to overfitting and reduced performance.

For color-only descriptors displayed in Figure 12, it is evident that 8 color bins were the optimal choice (e.g. globalRGBhisto_color_8bins) which achieved the highest MAP=0.166 compared to all other descriptors. This suggests that increasing even further the quantization level reduces discriminative power.

When spatial grids were applied to color histograms in Figure 12, the performance improved substantially compared to global color histograms without spatial information. Particullarly, by adding gridding to the 4 color bins, the descriptor 3x3_color_4bins reached mAP=0.183 and significantly surpassed the

globalRGBhisto_color_4bins descriptor that had mAP=0.157. This confirms that gridding the image adds valuable discriminative power.

For spatial-grid and edge descriptors in Figure 13, when the orientation bins increased, the performance also improved (e.g., 2x2_texture_16orient achieved MAP=0.199 compared to 2x2_texture_4orient at MAP=0.109). This confirms that implementing finer angular quantization enhances edge-based discrimination. However, by reviewing the confusion matrix diagonal for 3x3_texture_16orient in Figure 13 an uneven performance between classes can be spotted. In particular, the descriptor reached high predictions for classes 2,4,7,13,8,18,9,3,17 (tree, plane, car, bookselve, bicycle, boat, sheep, building, street) that have many edges but it had significantly lower predictions for classes 11,12,14,16,19,15 (sign, bird, bench, dog, people, cat) that have smoother regions. This indicates that edge orientation histograms are highly discriminative for images with many edges or structured scenes, but they may be insufficient for categories where color plays a more distinguishing role.

Descriptors that had spatial grid applied together with both color and edge histograms, showed good results as well, with the best descriptor being 3x3_both_4bins_8orient achieving mAP=0.194, followed by 2x2_both_4bins_8orient (MAP=0.185) and 2x2_both_8bins_8orient (mAP=0.184). While these had slightly less mAP compared the best edge orientation histogram descriptor (3x3_texture_16orient at mAP=0.204), comparing the confusion matrices in Figure 13 gives away important trade-offs. In contrast to the edge orientation histogram descriptor, the combined descriptor (3x3_both_4bins_8orient) demonstrated more balanced performance across all classes, achieving perfect probability for class 2 (1.00) and strong results for classes 4 (0.83), 13 (0.70), 17 (0.70), and 7 (0.63), while avoiding the extreme weaknesses seen in texture-only features by. This recommends that combining color information can help maintain the descriptor's discriminative ability across different scene categories, thus, making the combined descriptor more robust for practical image retrieval applications despite a slightly lower overall MAP score.

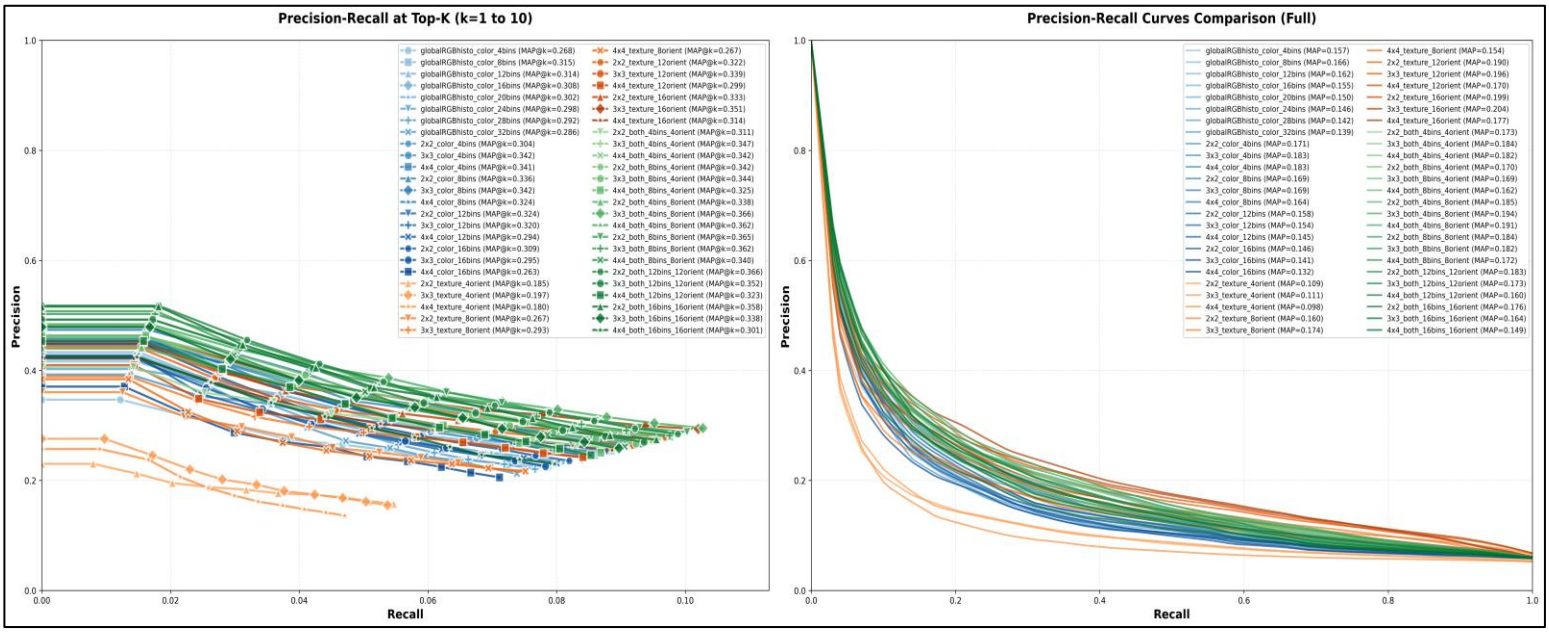

Figure 11: Comparison of averaged mAP of every class between all descriptors for top 10 results (left) and for all results (right) 46 experiments in total.

Figure 12: Comparison between best descriptors using only color histograms (top) and spatial grid with color histograms (bottom).

Figure 13: Comparison between best descriptors using spatial grid with edge histograms (top) and spatial grid with bot color histograms and edge histograms (bottom).

Use of PCA

Identifying the need of PCA

The techniques used so far include Global Color Histograms, Edge Orientation Histograms, and Spatial Grids which produce very high-dimensional vectors, and this can lead to several problems. While building all the previous steps of this image retrieval system the high dimensionality of feature descriptors caused high computational loads and time bottlenecks. It was verified that high-dimensional vectors increase computational complexity, making the system calculations significantly slower. These high-dimensional vectors also lead to the "curse of dimensionality" issue, where data are too sparse in the feature space and distance metrics fail to capture relations between them. In addition, some redundant or highly correlated features may dominate the similarity comparison, reducing retrieval accuracy. Therefore, the technique Principal Component Analysis (PCA) was applied on the system which helped reduce the dimensions of the feature space. This technique aided in keeping only the principal components that contained most of the information while removing redundant ones, therefore lowering the descriptor dimensions significantly.

Definition

PCA is a technique that is used to transform large data sets into fewer non-correlated variables (principal components) while retaining most of the original information. Thus, making visualization and analysis faster and more efficient [25]. PCA works by replacing the axes that represent the data with new ones that capture the maximum spread of data from the average value of them (variance). These new axes are called principal components, and each of them captures the next highest variance of data in the feature space.

The PCA process starts by subtracting the mean of each variable to center the data. It then calculates how variables co-vary with each other, using a covariance matrix, to identify correlations and redundant information. Afterwards, using this matrix it computes eigenvalues and eigenvectors. Computed eigenvectors define the directions of the new feature space's uncorrelated variables called principal components, while computed eigenvalues show each component's variance. Then, both are sorted in descending order of variance. Afterwards, it selects the top components that together explain a desired fragment of the total variance (e.g. 99%, 90%, 85%, ...) and discards the others. Lastly, it demonstrates the original data along the selected principal components to create a smaller dimensional representation [25].

Calculation procedure

To simulate how dimensionality reduction impacts the image's retrieval performance, an analysis was conducted on how reducing the dimensionality of image descriptors affects retrieval accuracy using Euclidean and Mahalanobis distance metrics. To do that the PCA technique was implemented using a custom Eigenmodel based on Eigenvalue Decomposition, optimized with SciPy's linear algebra routines [26]. In this method, the data matrix $X(samples \times features)$ is first calculated by subtracting the mean vector μ from each sample of X. The covariance matrix C is then computed as $C = \frac{1}{n-1}(X-\mu)^{\top}(X-\mu)$. Next, to improve computational efficiency for high-dimensional descriptors, instead of calculating the full

covariance matrix, a smaller matrix $X_{small} = XX^{\top}$of size $(samples \times samples)$ was decomposed into eigenvalues and eigenvectors. Afterwards, eigenvalues are sorted in descending order, and eigenvalues close to zero are removed to avoid numerical instability. Lastly, the full eigenvectors are reconstructed from these smaller eigenvectors, by multiplying the small matrix X_{small} with $X^{\top}$, and are then normalized.

To construct and apply PCA using the steps described above two files were structured, *run_pca_comparision.py* and *cvpr_pca_utils.py* that work together. Specifically, the function *compute_pca_projection()* takes as an input a variance threshold (0.7, 0.75, 0.8, 0.85, 0.90, 0.95, 0.99) and outputs the PCA model. This model contains the mean vector (μ), eigenvectors, eigenvalues, and explained variance ratios, the projected descriptors, obtained by projecting the mean-centered data onto the selected eigenvectors, and the fraction of variance explained by the selected components. Optionally, these PCA-reduced descriptors are saved for reproducibility.

For measuring descriptor similarity, two distance metrics are used. The first was the Euclidean distance which computes the distance between two feature vectors, ignoring correlations among features. The second was Mahalanobis distance which tracks similarity while considering for feature correlations. The function *compute_covariance_matrix()* computes the full covariance matrix (or a diagonal approximation for very high-dimensional data), adds a small regularization term ($1E^{-6}$) (to make sure that calculations don't output errors or unreliable results due to very small or very large numbers), and computes its inverse. This inverse covariance is then used to calculate the Mahalanobis distance $d_M(x, y) = \sqrt{(x-y)^{\top}\mathrm{Cov}^{-1}(x-y)}$, which penalizes deviations along directions of low variance more heavily [27].

After using a random image as a query, the system fetches the first 10 images from the total results, the precision and recall are computed for those 10 images and the retrieval performance is evaluated. In addition, the mAP was also computed as the mean of precision values across all queries. These calculations are implemented in *evaluate_experiment()*.

For an in-depth analysis, seven different configurations of PCA were evaluated in total using variance thresholds (70%, 75%, 80%, 85%, 90%, 95%, 99%) for both Euclidean and Mahalanobis distances and plotted against the two baselines without PCA accordingly. Within *run_all_experiments()*, PCA is applied where specified, inverse covariance matrices are computed for Mahalanobis experiments, and retrieval evaluations are performed for each configuration. The results, including mAP, P@K, R@K, and dimensionality, are stored and visualized using *plot_comparison_results()* and the computed plots include comparisons of mAP versus dimensionality, precision and recall curves, efficiency plots of P@10 versus dimensionality, and percentage improvement over baseline methods. Lastly, final numerical results are saved and displayed in *save_results_table()*, which produces a CSV file and highlights the best-performing configuration.

Insights

After evaluating multiple descriptor configurations to investigate interaction between PCA dimensionality reduction and Mahalanobis distance metrics two of them were selected for comparison. Specifically, spatialGrid_2x2_both_16bins_16orient (16448 dimensions) and spatialGrid_4x4_both_16bins_16orient (65792 dimensions) descriptors were selected on purpose to compare the effectiveness of PCA with Mahalanobis distance across different descriptor sizes.

The results were plotted as shown in Figure 14 and the following information can be extracted for each descriptor:

For the 2x2 grid in Figure 14 (16,448 baseline dimensions), when using Euclidean distance with PCA of 70% variance, the feature space is reduced to just 56 dimensions (a 99.7% reduction), and achieves the highest mAP@[1-max], outperforming even the full baseline. However, as more variance is retained (moving from 70% to 99%), performance decreases, with PCA 99% (406 dimensions) dropping way below the baseline. This result indicates that the most discriminative information for Euclidean distance is concentrated in the top 56 principal components, while the remaining ~16,000 dimensions mainly contribute noise and redundant correlations. The same pattern continues for Mahalanobis distance, while its baseline (16,448 dimensions) achieves smaller mAP than Euclidiean's baseline, and drops to half of original mAP for PCA 99%, it drastically improves when applying PCA 70%. Here, the best option is keeping 70–85% variance, where the covariance matrix remains well-conditioned and invertible.

For the 4x4 grid in Figure 14 (65,792 baseline dimensions, four times larger than 2x2), the same patterns appear. Euclidean distance performs poorly in full dimensions but improves significantly under compression with PCA 70% (112 dimensions, 99.8% reduction) nearly matching the 2x2 results despite the finer spatial resolution. But again, the performance drops when moving from PCA 75% to PCA 99%, losing 26% of the relative performance when retaining more components. Mahalanobis distance again shows low results in this high-dimensional space the baseline (65,792 dimensions), improves PCA 70% but doesn't surpass Euclidean's baseline, and then drops significantly when applying PCA 99%. This decline demonstrates the increasing difficulty of accurately estimating and inverting a large covariance matrix.

Precision and recall metrics across K values reinforce these observations since Euclidean distance, in both spatial grids achieve higher precision and recall compared to Mahalanobis distance.

Overall, the optimal configuration is clearly PCA 70% with Euclidean distance on either spatial grid, compressing 16,448 dimensions to 56 (2x2) or 65,792 to 112 (4x4), achieving considerably higher mAP@[1-10] and mAP@[1-max] scores while drastically reducing storage and computation. Also, the finer 4x4 grid does not provide a performance advantage over 2x2. Lastly, Mahalanobis distance is only competitive under extreme compression (70–75% variance) but never surpasses Euclidean performance in this system. Therefore, Mahalanobis distance should always be paired with PCA.

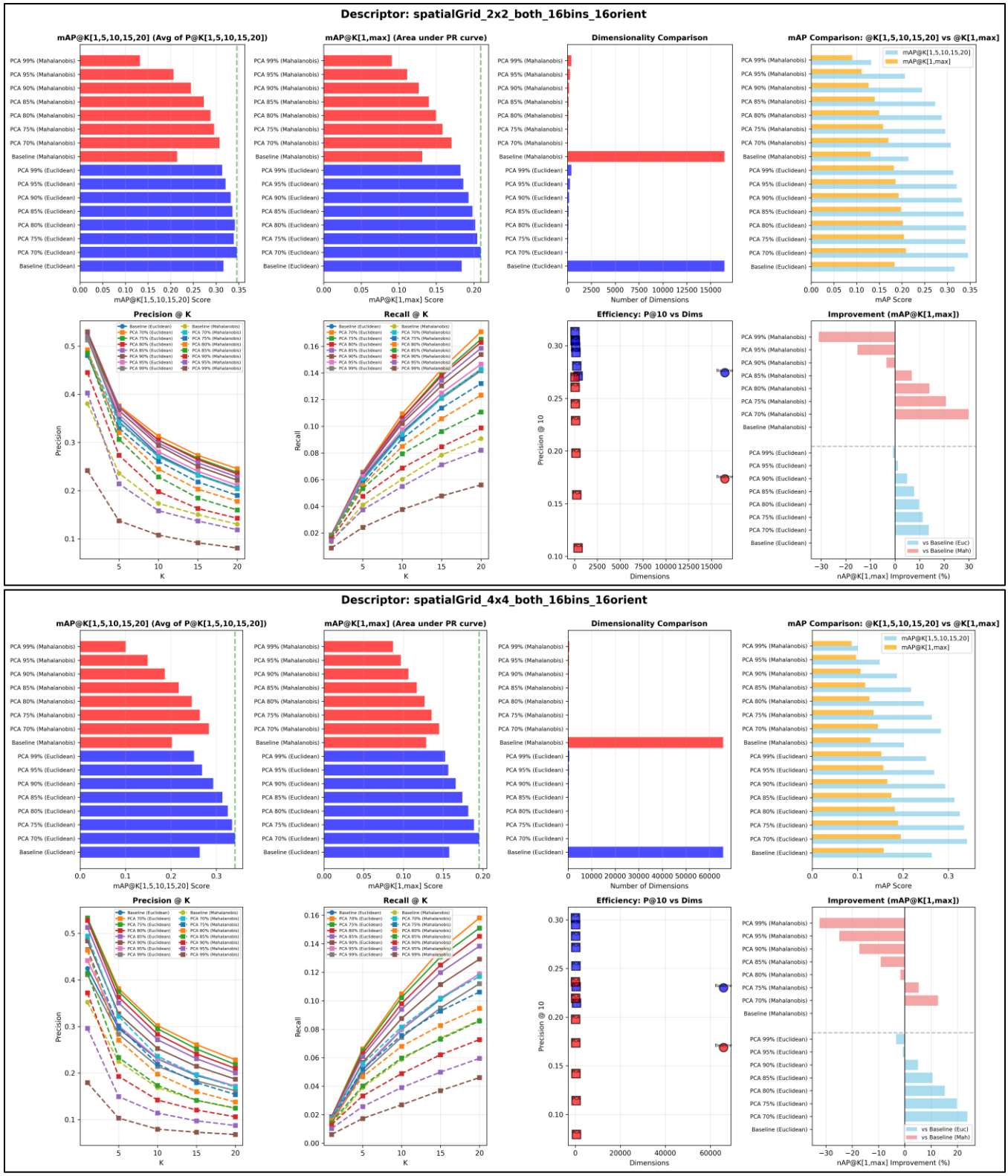

Figure 14: PCA experiments for mAP, dimensionality comparison, mAP@K comparison, P@K[1,5,10,15,20], R@K[1,5,10,15,20], P@10 vs dimentionality, and mAP % improvment over baselines for spatialGrid_2x2_both_16bins_16orient descriptor (top) and spatialGrid_4x4_both_1

In the Table 3 below, it is shown in column “dimensions” that the dimensionality has been significantly reduced successfully for every PCA experiment! Also, by analyzing the column “mAP@K” we can see that the mean average precision can be sustained and even improved when retaining specific percentages of the total variance in the descriptor! Specifically, less than 95% for smaller or larger dimensions and Euclidean distance, less than 95% for smaller dimensions and Mahalanobis distance, and less than 85% for larger dimensions and Mahalanobis distance.

	pca_comparison_results_spatialGrid_2x2_both_16bins_16orient			pca_comparison_results_spatialGrid_4x4_both_16bins_16orient		
Euclidean	**pca**	**dimensions**	**mAP@K**	**pca**	**dimensions**	**mAP@K**
Baseline	False	16448	0.3163	False	65792	0.2634
PCA 70%	True	56	0.3459	True	112	0.3411
PCA 75%	True	72	0.3392	True	139	0.3346
PCA 80%	True	93	0.3414	True	174	0.3250
PCA 85%	True	123	0.3362	True	218	0.3131
PCA 90%	True	168	0.3323	True	279	0.2930
PCA 95%	**True**	**246**	**0.3209**	**True**	**371**	**0.2686**
PCA 99%	True	406	0.3134	True	509	0.2508
Mahalanobis	**pca**	**dimensions**	**mAP@K**	**pca**	**dimensions**	**mAP@K**
Baseline	False	16448	0.2143	False	65792	0.2021
PCA 70%	True	56	0.3075	True	112	0.2836
PCA 75%	True	72	0.2957	True	139	0.2636
PCA 80%	True	93	0.2878	True	174	0.2460
PCA 85%	True	123	0.2731	**True**	**218**	**0.2175**
PCA 90%	**True**	**168**	**0.2444**	True	279	0.1867
PCA 95%	True	246	0.2063	True	371	0.1488
PCA 99%	True	406	0.1321	True	509	0.1005

Table 3: Different levels of PCA using Euclidean and Mahalanobis distances on pca_comparison_results_spatialGrid_2x2_both_16bins_16orient descriptor and pca_comparison_results_spatialGrid_4x4_both_16bins_16orient descriptor.

Other distance measures

It is also important to find out which distance measures can perform better in a visual search system like this one in order to further improve its effectiveness. To do that, other distance metrics, that have been identified to work well in scenarios like this one, will be put to test and compared with each other to evaluate their performance.

Some popular distance metrics that have been found to work well when evaluating similarity between images, are: the *Manhattan distance (L_1 Norm)*, the *Euclidean distance (L_2 Norm)*, and the *Mahalanobis distance* [20]. Further distances for comparing histograms of images include the *Correlation distance*, the *Chi-Square distance*, the *Histogram Intersection distance*, and the *Bhattacharyya distance* [28], [29]. Also, *Minkowski distance* used for pattern recognition, similarity between images, and feature detection [30], *Chebyshev distance* used for edge detection or pattern recognition [31], *Cosine distance* used for object's stance/pose comparison (by using points and joints) and approximation in images in computer vision tasks [32], it also helps with sorting out faces in a crowd or classifying nature photos [33]. Furthermore, *Canberra distance* is used for object recognition [34] as well as image recognition [35] and *Hellinger distance* used to compare image histograms [36].

Calculation procedure

These distance metrics were implemented in *cvpr_compare.py* using the following mathematical formulas:

- **Manhattan Distance (L_1 Norm):** $d_{manhattan}(F_1, F_2) = \sum_{i=1}^{n} | F_{1,i} - F_{2,i} |$, [37].
- **Euclidean Distance (L_2 Norm):** $d_{euclidean}(F_1, F_2) = \sqrt{\sum_{i=1}^{n}(F_{1,i} - F_{2,i})^2}$, [38].
- **Minkowski Distance (L_p Norm):** $d_{minkowski}(F_1, F_2) = (\sum_{i=1}^{n} | F_{1,i} - F_{2,i} |^p)^{\frac{1}{p}}$, [39].
- **Chebyshev Distance ($L\infty$ Norm):** $d_{chebyshev}(F_1, F_2) = \max_i | F_{1,i} - F_{2,i} |$, [40].
- **Cosine Distance:** $d_{cosine}(F_1, F_2) = 1 - \frac{F_1 \cdot F_2}{\|F_1\| \|F_2\|}$, [41].
- **Canberra Distance:** $d_{canberra}(F_1, F_2) = \sum_{i=1}^{n} \frac{|F_{1,i} - F_{2,i}|}{|F_{1,i}| + |F_{2,i}|}$, [42].
- **Mahalanobis Distance:** $d_{mahalanobis}(F_1, F_2) = \sqrt{(F_1 - F_2)^T \, Cov^{-1} \, (F_1 - F_2)}$, [43], [44].
- **Chi-Square Distance:** $d_{\chi^2}(F_1, F_2) = \frac{1}{2}\sum_{i=1}^{n} \frac{(F_{1,i} - F_{2,i})^2}{F_{1,i} + F_{2,i}}$, [27].
- **Histogram Intersection Distance:** $d_{intersection}(F_1, F_2) = 1 - \sum_{i=1}^{n} \min(F_{1,i}, F_{2,i})$, [28].
- **Bhattacharyya Distance:** $d_{bhattacharyya}(F_1, F_2) = -\ln(\sum_{i=1}^{n} \sqrt{F_{1,i} F_{2,i}})$, [29].
- **Hellinger Distance:** $d_{hellinger}(F_1, F_2) = \sqrt{\frac{1}{2}\sum_{i=1}^{n}(\sqrt{F_{1,i}} - \sqrt{F_{2,i}})^2}$, [45].
- **Correlation Distance:** $d(H_1, H_2) = \frac{\sum_i (H_1(i) - \overline{H_1})(H_2(i) - \overline{H_2})]}{\sqrt{[(\sum_i (H_1(i) - \overline{H_1})^2)(\sum_i (H_2(i) - \overline{H_2})^2}}$, [28].

The system uses *plot_distance_metric_comparison()* to analyze all metrics with bar charts displaying mAP@[1-max] (mean of full precision curve) and mAP@[1-10] (mean precision at top-10 ranks).

Insights

The performance evaluation of twelve distance metrics was conducted using Mean Average Precision (mAP) across multiple feature descriptor combinations, measuring both top-10 mAP (top-10 ranked results) and full mAP (overall retrieval performance). Both plots that are displayed in Figure 15 have the same patterns.

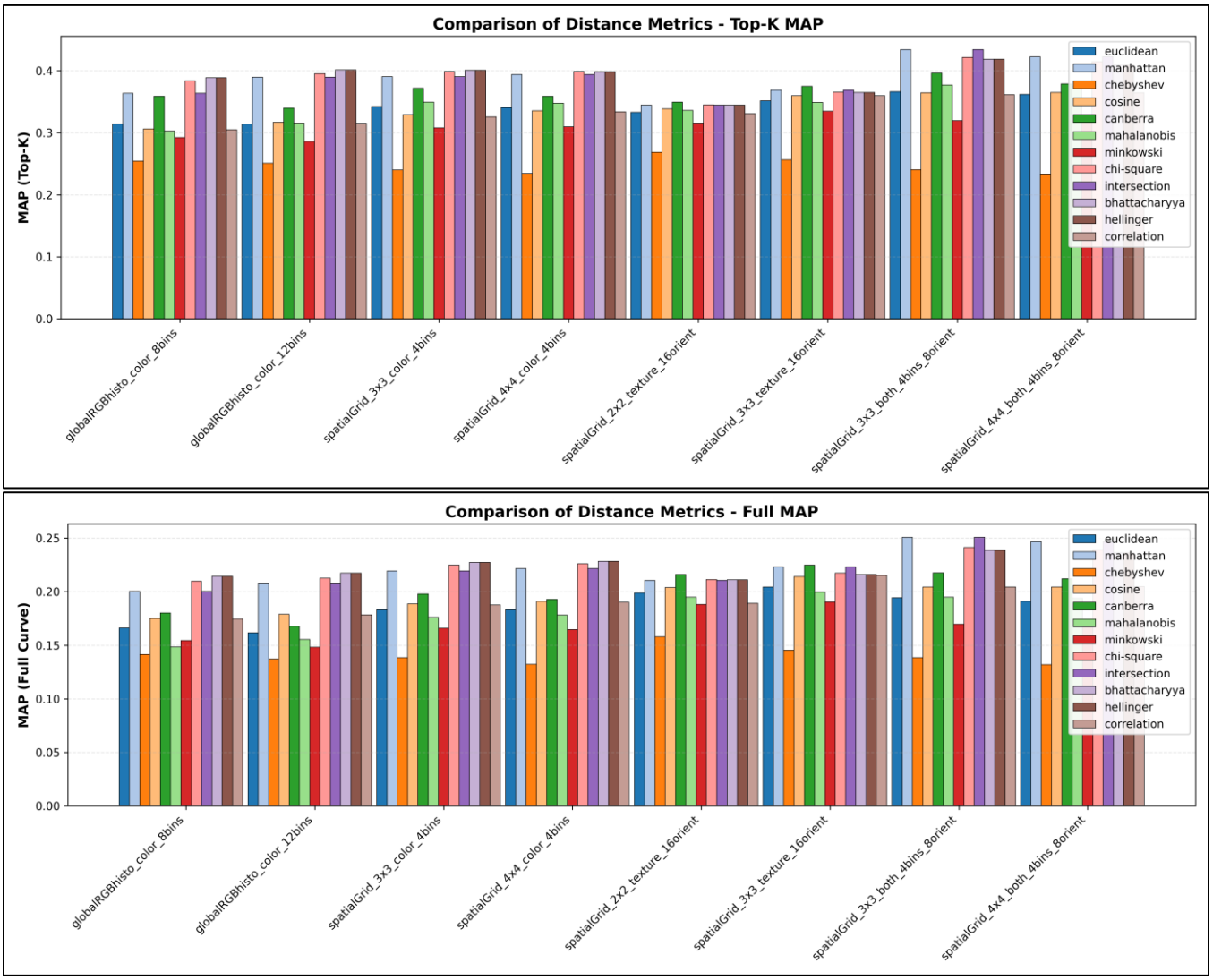

Figure 15: mAP@K[1-10] & mAP@K[1-max] for different 8 descriptors across 12 different distance metrics.

Specifically, the results show a clear performance difference among the distance metrics. Bhattacharyya, Hellinger, Chi-Square, Manhattan and Intersection distances consistently achieved the highest performance in terms of mAP scores. In addition, Manhattan and Intersection distances reached

significantly higher results in descriptors that combined spatial grid with both color bins and angular quantization bins, making them the best choice for descriptors with higher dimensions.

Lastly, as descriptors grew in dimensionality Bhattacharyya, Hellinger, Chi-Square, Manhattan and Intersection distances figured better mAP results for them. This suggests that further experimentation using these distances with more descriptors, both larger and computational heavy (e.g. spatialGrid_4x4_both_4bins_8orient) or smaller descriptors with less features (e.g. spatialGrid_3x3) can help achieve even higher mAP scores.

To demonstrate the results of the most accurate descriptor spatialGrid_3x3_both_4bins_8orient, different query results for the top 10 retrieved images of the same query from class 2 (tree-class) are showcased in Figure 16 using Bhattacharyya, Hellinger, Chi-Square, Manhattan and Intersection distances for visual comparison of the results.

As expected, by using the tree-class that contains images that have similar colors (green and brown), edges (leaves, and bunches) and spatial information (grass, trees), combined with the best distance metrics the results are near perfect. It can also be noticed that for some distances the system also retrieves an image from class 17 (city roads) that contains a tree, this indicates that the system is learning meaningful visual features rather than overfitting to class 2. This shows generalization capability as it is recognizing semantic content (trees) across different backgrounds and surroundings, not just memorizing the specific characteristics of class 2.

Figure 16: spatialGrid_3x3_both_4bins_8orient with Bhattacharyya (top-left), Hellinger (top-right), Chi-Square (middle-left), Manhattan (middle-right), Intersection (bottom-left).

References

[1] GarageFarm.net, "Color Histograms Demystified: From Theory to Practical Image Analysis," 2025. [Online]. Available: https://garagefarm.net/blog/color-histograms-demystified-from-theory-to-practical-image-analysis.

[2] R. K, "A Comprehensive Guide on LLM Quantization and Use Cases," Analytics Vidhya, 13 August 2024. [Online]. Available: https://www.analyticsvidhya.com/blog/2024/08/llm-quantization/.

[3] S. N. S. Sung-Hyuk Chaa, "On measuring the distance between histograms," *PATTERN RECOGNITION - THE JOURNAL OF THE PATTERN RECOGNITION SOCIETY,* p. 16, 4 June 2001.

[4] A. Rosebrock, "How-To: 3 Ways to Compare Histograms using OpenCV and Python," pyimagesearch.com, 14 July 2024. [Online]. Available: https://pyimagesearch.com/2014/07/14/3-ways-compare-histograms-using-opencv-python/.

[5] C. A.-R. a. B. T. G. Manuel G. Forero, "Analytical Comparison of Histogram Distance," *Facultad de Ingeniería, Universidad de Ibagué, Ibagué, Colombia,* p. 10, 2019-03-03.

[6] GeeksforGeeks, "Calculate the Euclidean distance using NumPy," geeksforgeeks.org, 15 July 2025. [Online]. Available: https://www.geeksforgeeks.org/python/calculate-the-euclidean-distance-using-numpy/.

[7] R. a. C. V. Malini, "COLOR PERCEPTION HISTOGRAM FOR IMAGE RETRIEVAL USING MULTIPLE SIMILARITY MEASURES," *Department of Electronics and Communication Engineering, Government College of Technology, Coimbatore, Tamilnadu, India,* p. 10, 2014-01-30.

[8] Y. J. V. R. D. Latha, "Hybrid CBIR method using statistical, DWT-Entropy and POPMV-based feature sets," *IET Image Processing,* 2019.

[9] Wikipedia, "Precision and recall," Wikipedia, 14 October 2025. [Online]. Available: https://en.wikipedia.org/wiki/Precision_and_recall.

[10] A. a. G. P. Deshmukh, "An improved content based image retreival," *2011 2nd International Conference on Computer and Communication Technology (ICCCT-2011),* 2011.

[11] Scikit-Learn, "Precision-Recall," Scikit-Learn, 2025. [Online]. Available: https://scikit-learn.org/stable/auto_examples/model_selection/plot_precision_recall.html.

[12] Yellowbrick, "Yellowbrick: Machine Learning Visualization," Yellowbrick, 2019. [Online]. Available: https://www.scikit-yb.org/en/latest/.

[13] how.dev, "What is the mean average precision in information retrieval?," how.dev, 2025. [Online]. Available: https://how.dev/answers/what-is-the-mean-average-precision-in-information-retrieval.

[14] P. R. H. S. Christopher D. Manning, "8.4 Evaluation of ranked retrieval results," in *An Introduction to Information Retrieval*, England, Cambridge University Press, Cambridge, England, April 1, 2009, p. 581.

[15] Google, "Google Landmark Retrieval 2019," kaggle.com, 4 June 2019. [Online]. Available: https://www.kaggle.com/competitions/landmark-retrieval-2019.

[16] Wikipedia, "Evaluation measures (information retrieval)," Wikipedia, 17 August 2025. [Online]. Available: https://en.wikipedia.org/wiki/Evaluation_measures_%28information_retrieval%29.

[17] Wikipedia, "Confusion matrix," Wikipedia, 23 October 2025. [Online]. Available: https://en.wikipedia.org/wiki/Confusion_matrix.

[18] V. Jayaswal, "Performance Metrics: Confusion matrix, Precision, Recall, and F1 Score," Towards Data Science, 14 September 2020. [Online]. Available: https://towardsdatascience.com/performance-metrics-confusion-matrix-precision-recall-and-f1-score-a8fe076a2262/.

[19] R. Kundu, "Confusion Matrix: How To Use It & Interpret Results [Examples]," v7labs.com, 13 September 2022. [Online]. Available: https://www.v7labs.com/blog/confusion-matrix-guide.

[20] M. Boels, "Visual Search for Image Retrieval, Images Descriptors and Detectors," medium.com, 31 January 2020. [Online]. Available: https://medium.com/@boelsmaxence/visual-search-for-image-retrieval-2c8cafc792fe.

[21] C. S. a. J. P. Svetlana Lazebnik, "Beyond Bags of Features: Spatial Pyramid Matching," p. 8, 2005.

[22] N. D. a. B. Triggs, "Histograms of Oriented Gradients for Human Detection," *IEEE Xplore,* 25 July 2005.

[23] OpenCV, "Sobel Derivatives," Open Source Computer Vision, 17 June 2025. [Online]. Available: https://docs.opencv.org/3.4/d2/d2c/tutorial_sobel_derivatives.html.

[24] Wikipedia, "Histogram of oriented gradients," Wikipedia, 12 March 2025. [Online]. Available: https://en.wikipedia.org/wiki/Histogram_of_oriented_gradients.

[25] Z. Jaadi, "Principal Component Analysis (PCA): A Step-by-Step Explanation," builtin.com, 23 June 2025. [Online]. Available: https://builtin.com/data-science/step-step-explanation-principal-component-analysis.

[26] scipy.org, "SciPy 1.16.2," scipy.org, 11 September 2025. [Online]. Available: https://scipy.org/.

[27] N. U. B. a. P. N. V. Asha, "GLCM based Chi-square Histogram Distance for," *IJCVR, Vol. 2, No. 4, 2011, pp. 302-313,* vol. Vol. 2, p. 7, 2011.

[28] OpenCV, "Histogram Calculation," OpenCV, 17 June 2025. [Online]. Available: https://docs.opencv.org/3.4/d8/dc8/tutorial_histogram_comparison.html.

[29] K. Safjan, "Understanding Bhattacharyya Distance and Coefficient for Probability Distributions," Krystian Safjan's Blog, 30 June 2025. [Online]. Available: https://safjan.com/understanding-bhattacharyya-distance-and-coefficient-for-probability-distributions/.

[30] V. Chugani, "Minkowski Distance: A Comprehensive Guide," datacamp.com, 9 October 2024. [Online]. Available: https://www.datacamp.com/tutorial/minkowski-distance.

[31] V. Chugani, "Understanding Chebyshev Distance: A Comprehensive Guide," https://www.datacamp.com/, 5 September 2024. [Online]. Available: https://www.datacamp.com/tutorial/chebyshev-distance.

[32] R. Alake, "Understanding Cosine Similarity and Its Applications," builtin.com, 5 March 2025. [Online]. Available: https://builtin.com/machine-learning/cosine-similarity.

[33] V. Chugani, "What is Cosine Distance?," datacamp.com, 28 July 2024. [Online]. Available: https://www.datacamp.com/tutorial/cosine-distance.

[34] K. Kumar, "Object Recognition Using Shape Context with Canberra Distance," *Asian Journal of Applied Science and Technology (AJAST),* vol. 13, p. March, 2017.

[35] S. Eskandar, "Exploring Common Distance Measures for Machine Learning and Data Science: A Comparative Analysis," medium.com, 21 March 2023. [Online]. Available:

https://medium.com/@eskandar.sahel/exploring-common-distance-measures-for-machine-learning-and-data-science-a-comparative-analysis-ea0216c93ba3.

[36] P. D. Wilson, "Distance-based methods for the analysis of maps produced by species distribution models," © 2011 The Author Methods in Ecology and Evolution © 2011 British Ecological Society, 2 June 2011. [Online]. Available: https://besjournals.onlinelibrary.wiley.com/doi/full/10.1111/j.2041-210X.2011.00115.x.

[37] Wikipedia, "Taxicab geometry," Wikipedia, 23 September 2025. [Online]. Available: https://en.wikipedia.org/wiki/Taxicab_geometry.

[38] Wikipedia, "Euclidean distance," Wikipedia, 8 September 2025. [Online]. Available: https://en.wikipedia.org/wiki/Euclidean_distance.

[39] Wikipedia, "Minkowski distance," Wikipedia, 11 September 2025. [Online]. Available: https://en.wikipedia.org/wiki/Minkowski_distance.

[40] Wikipedia, "Chebyshev distance," Wikipedia, 2 October 2025. [Online]. Available: https://en.wikipedia.org/wiki/Chebyshev_distance.

[41] Wikipedia, "Cosine similarity," Wikipedia, 17 September 2025. [Online]. Available: https://en.wikipedia.org/wiki/Cosine_similarity.

[42] Wikipedia, "Canberra distance," Wikipedia, 30 March 2024. [Online]. Available: https://en.wikipedia.org/wiki/Canberra_distance.

[43] Y. Dodge, The Concise Encyclopedia of Statistics, New York: Springer, 2008, p. 612.

[44] Wikipedia, "Mahalanobis distance," Wikipedia, 29 September 2025. [Online]. Available: https://en.wikipedia.org/wiki/Mahalanobis_distance.

[45] HandWiki.org, "Hellinger distance," HandWiki.org, 6 February 2024. [Online]. Available: https://handwiki.org/wiki/Hellinger_distance.